

SNAP Qube Powered by Drayvn AI

Product Narrative and Jira-Oriented Implementation Plan

Field	Value
Document Purpose	Management and development planning report
Audience	Executive stakeholders, product leadership, software architecture, engineering, QA, field operations
Prepared For	Procal Technologies, Inc.
Date	May 8, 2026
Status	Polished planning draft for review and execution alignment

26

Jira Epics

284

Implementation Stories

6

RBAC Roles

5

Release Waves

Positioning Statement

SNAP Qube is a real-time, AI-assisted RAN intelligence workbench for planning, executing, analyzing, replaying, and reporting application-oriented mobile network drive tests using autonomous Android devices.

Table of Contents

1. Executive Summary

4

2. Product Narrative

4

2.1 Product Purpose

5

2.2 Product Differentiation

5

2.3 Operating Model

5

3. Core Product Domains

5

4. High-Level System Architecture

6

4.1 Major Components

7

4.2 Live Execution Flow

7

4.3 Drayvn AI IQ Agent Architecture

8

5. Recommended Data Architecture

8

6. RBAC and Visibility Model

8

6.1 Role Responsibilities

9

6.2 Role Scope Matrix

9

7. React Qube Portal Experience

9

8. Jira Epic Portfolio

10

9. Detailed Jira Implementation Plan

11

SQ-001: Product Foundation and System Architecture

11

SQ-002: Team, Subteam, and Tenant Hierarchy Management

11

SQ-003: Identity, Authentication, Authorization, and RBAC

11

SQ-004: Autonomous Android Device Registry

12

SQ-005: Autonomous Android Controller Command Plane

12

SQ-006: APK Lifecycle, OTA Updates, and Device Log Retrieval

13

SQ-007: Drive Plan Authoring and CRUD

13

SQ-008: RAN Infrastructure Inventory and Map Layers

13

SQ-009: Test Profile and Application Workload Catalog

14

SQ-010: Live Drive Execution Orchestration	14
SQ-011: Real-Time Telemetry Ingestion	15
SQ-012: Real-Time Streaming to the Qube Portal	15
SQ-013: React Qube Portal Foundation	15
SQ-014: Planner Experience and Drive Plan Canvas	16
SQ-015: Tester Live Execution Experience	16
SQ-016: RAN and Application Scoring Engine	16
SQ-017: Drayvn AI IQ Layer and Agentic Analysis	17
SQ-018: Post-Execution Reporting	17
SQ-019: Drive Plan Replay	18
SQ-020: Alerts, Notifications, and Operational Exceptions	18
SQ-021: Backend API Gateway and Service Contracts	19
SQ-022: Data Governance, Security, and Audit	19
SQ-023: Observability, Monitoring, and Operations	19
SQ-024: Automated Testing and Verification	20
SQ-025: Deployment, CI/CD, and Environment Management	20
SQ-026: Documentation, Training, and Operational Readiness	20
10. Suggested Release Plan	21
11. Global Definition of Done	21
12. Critical Product and Architecture Decisions	22
12.1 AI Guardrails	22
13. MVP Scope Recommendation	22
14. Recommended Next Actions	23

1. Executive Summary

SNAP Qube powered by Drayvn AI is an application-oriented mobile Radio Access Network analysis platform designed to modernize live drive testing. The system combines autonomous Android devices, a field controller, real-time telemetry ingestion, geospatial RAN visualization, application-level test orchestration, replay, reporting, and evidence-bound AI analysis.

The result is a unified operational workflow for constructing Drive Plans, executing them in the field, visualizing live RAN and application behavior, and generating actionable post-execution intelligence for engineering and management stakeholders.

2

Primary Product Purposes

26

Capability Epics

284

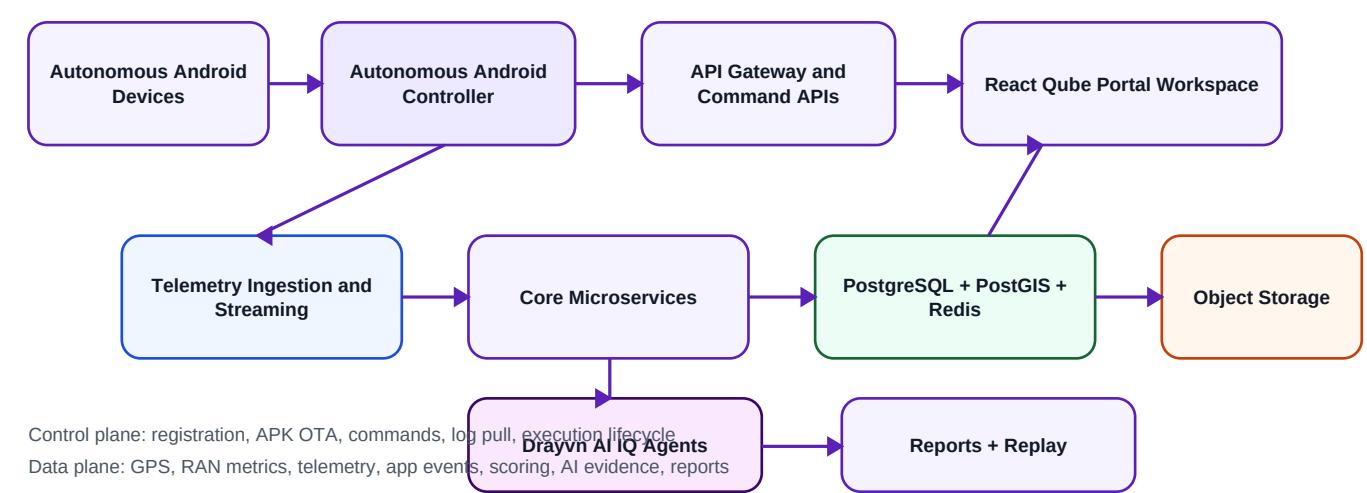
Jira Implementation Stories

12

Recommended MVP Capabilities

Management View	Development View
A commercially credible AI-assisted drive-test platform for RAN engineering, application experience evaluation, and operational reporting.	A microservice-oriented ecosystem with React workspace UX, Android APK integration, controller command plane, telemetry pipeline, PostGIS data model, and Drayvn AI analysis layer.
The differentiator is real-time visibility plus post-run AI explanation, not merely data capture.	The core implementation risk is coordination across device control, telemetry scale, geospatial modeling, RBAC scope, and AI evidence binding.

SNAP Qube Ecosystem Architecture



Primary Recommendation
Build SNAP Qube around PostgreSQL/PostGIS as the authoritative data platform, a React canvas-oriented workspace for the portal, and Drayvn AI agents constrained by evidence-bound analysis rules.

2. Product Narrative

SNAP Qube powered by Drayvn AI is an application-oriented mobile RAN analysis platform designed to transform live drive testing from a manually coordinated, fragmented, post-processed engineering activity into a real-time, autonomous, AI-assisted operational workflow.

SNAP Qube uses autonomous Android devices running the Drayvn Autonomous Android APK to collect radio access network metrics, device telemetry, GPS positioning, application behavior, and end-user experience indicators during live drive tests. These devices are remotely registered, controlled, updated, commanded, and monitored by the Autonomous

Android Controller, which acts as the operational control plane for field execution.

The system is designed for two primary purposes: construction of Drive Plans and analysis of RAN metrics and telemetry captures while performing end-user tests for calling, video playback, music playback, SMS/MMS texting, upload/download performance, and general web browser responsiveness.

2.1 Product Purpose

Purpose	Description
Drive Plan Construction	Network planners create structured drive plans for specific roadways, regions, service areas, RAN infrastructure clusters, antenna sectors, or areas of known concern. Plans define where to drive, which devices to use, what tests to run, what metrics to collect, what infrastructure to observe, and what success thresholds must be evaluated.
Real-Time RAN and Application Experience Analysis	During execution, testers coordinate field technicians driving vehicles along planned routes. The portal visualizes real-time GPS movement, RAN antenna overlays, serving-cell behavior, application test status, KPI scores, degradation, coverage holes, throughput trends, call/SMS/MMS behavior, media playback quality, and browser responsiveness.

2.2 Product Differentiation

SNAP Qube is not merely a drive-test portal. It is an AI-assisted RAN intelligence workbench. Drayvn AI agents analyze telemetry, detect anomalies, correlate network behavior with application experience, explain degradation events, generate execution summaries, and assist engineering users in distinguishing coverage issues, handoff instability, interference, congestion, poor throughput, device state problems, application behavior, and execution anomalies.

2.3 Operating Model

Concept	Definition
Team	A top-level organizational boundary, such as a national wireless network organization.
Subteam	A region, market, district, engineering group, or nested operational domain under a Team or parent Subteam.
Scope	Devices, users, drive plans, executions, reports, and visibility are constrained by Team/Subteam hierarchy.
Controller	The Autonomous Android Controller accepts device registrations, controls APK updates, pulls APK logs, initiates/terminates drive-plan telemetry, and provides real-time streams to the Qube portal.

3. Core Product Domains

The product domain is centered on organizational hierarchy, autonomous devices, planned routes, RAN infrastructure, live execution, telemetry events, application tests, replayable evidence, AI findings, and reports.

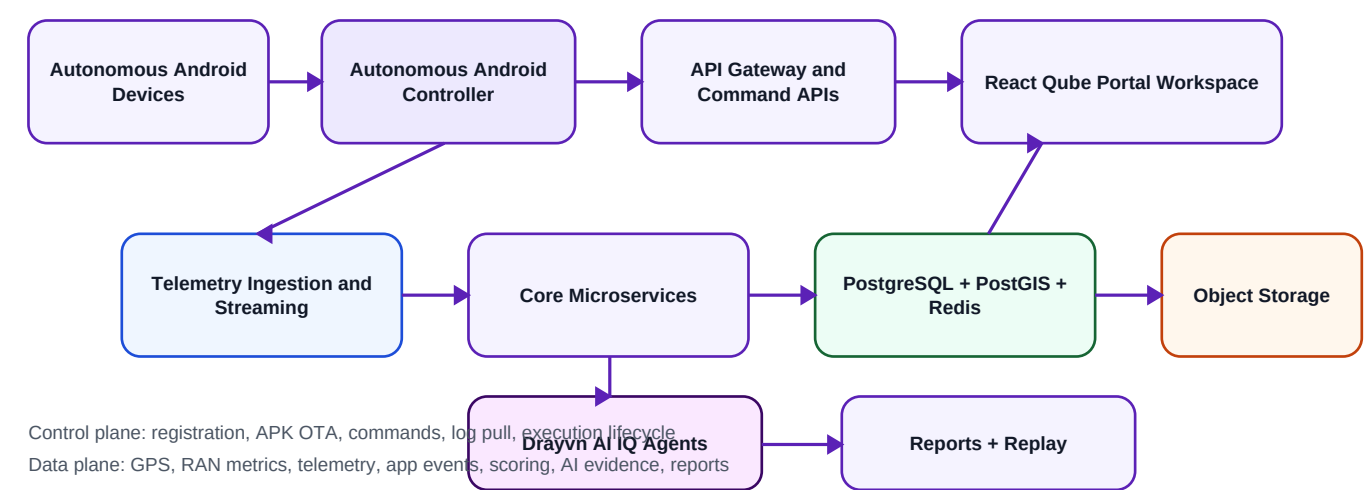
Domain Object	Description
Team	Top-level organizational boundary
Subteam	Recursive child organization under a Team or parent Subteam
User	Human portal user assigned roles and visibility scope
Role	Functional permission bundle
Device	Registered Autonomous Android device
Device Assignment	Association between device, Team/Subteam, test pool, and operational status
APK Version	Managed release of the Drayvn Autonomous Android APK
Drive Plan	Planned route, area, infrastructure focus, test suite, devices, schedule, and thresholds
Route	Ordered geospatial path with waypoints and roadway metadata
Area of Concentration	Polygon, corridor, market area, or manually selected map area
RAN Infrastructure Asset	Antenna, site, sector, cell, gNodeB/eNodeB, carrier, band, PCI, TAC, NR/LTE attributes
Test Profile	Calling, SMS/MMS, music playback, video playback, browser, upload/download, latency, responsiveness
Execution Session	One live or completed run of a drive plan
Telemetry Event	GPS, radio metrics, device state, app test event, throughput, latency, failure, handoff event

Domain Object	Description
Live Stream	Real-time data channel from device/controller to portal
Replay Track	Reconstructable execution timeline for post-run review
AI Analysis	Agent-generated insight, anomaly explanation, summary, recommendations
Report	Human-readable post-execution summary with charts, maps, scores, and evidence
Artifact	APK logs, execution logs, packet summaries, screenshots, video snippets, exported reports

4. High-Level System Architecture

SNAP Qube should be implemented as a set of well-formed microservices, each exposing clear APIs and owning specific data responsibilities. The Qube portal consumes these services through the API gateway and live streaming layer. The Autonomous Android Controller owns the field device command plane.

SNAP Qube Ecosystem Architecture

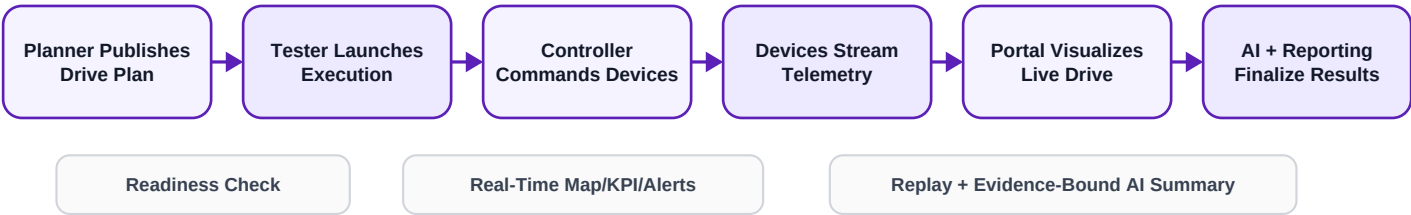


4.1 Major Components

Component	Responsibility
React Qube Portal	Role-specific UI, drive-plan authoring, live execution, map, replay, dashboards, reports
API Gateway	Auth enforcement, routing, throttling, API versioning, request auditing
Identity/RBAC Service	Users, roles, scopes, team/subteam permissions
Team/Subteam Service	Recursive org hierarchy and visibility boundaries
Device Registry Service	Device enrollment, configuration capture, assignment, status
Autonomous Android Controller	Device control plane, command dispatch, APK update control, log pull, session lifecycle
OTA Service	APK version catalog, staged rollout, forced upgrade, rollback
Drive Plan Service	CRUD, versioning, approval, route geometry, test profile binding
RAN Infrastructure Service	Antenna/site/sector inventory, map overlays, geospatial querying
Execution Orchestration Service	Starts/stops drive plans, binds devices, coordinates tests, tracks session state
Telemetry Ingestion Service	Receives high-volume device telemetry and RAN metrics
Real-Time Streaming Service	Pushes live data to portal via WebSocket/SSE
Scoring/KPI Service	Computes application-oriented RAN and UX quality scores
Replay Service	Reconstructs execution timeline and map movement
Reporting Service	Generates execution summaries, charts, exports
Drayvn AI IQ Layer	Agentic analysis, anomaly detection, recommendations, summaries
Artifact Service	Stores and retrieves logs, APK logs, exports, replay artifacts
Notification Service	Alerts for device failure, route deviation, test failure, KPI threshold breach
Audit Service	Tracks user, device, controller, and AI actions
Observability Stack	Metrics, logs, traces, operational dashboards

4.2 Live Execution Flow

Live Drive Execution Flow

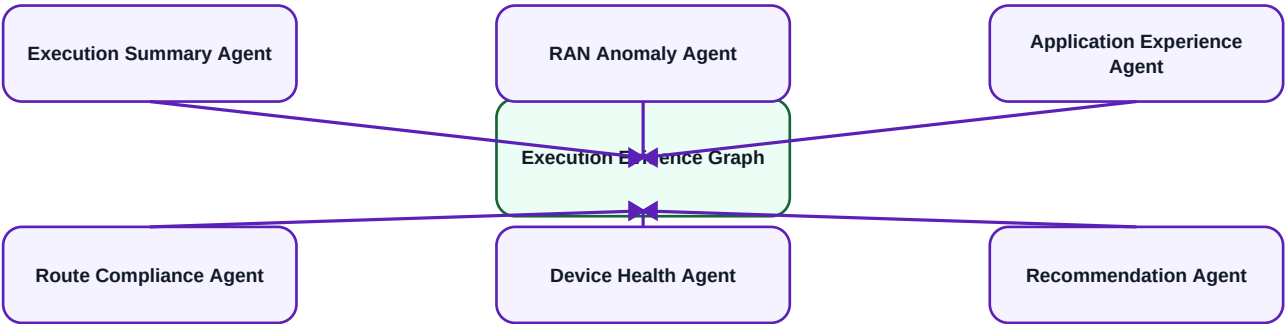


Execution state, route adherence, test status, RAN metrics, telemetry quality, and device health are tracked as first-class evidence.

The execution model begins with a published Drive Plan, advances through readiness checks, controller-directed device execution, live telemetry streaming, portal visualization, and final post-execution analysis. This flow must be deterministic, auditable, and replayable.

4.3 Drayvn AI IQ Agent Architecture

Drayvn AI IQ Layer - Agent Responsibilities



All AI findings must bind to telemetry, KPI, map segment, artifact, or infrastructure evidence and include confidence/uncertainty.

The AI layer should operate against a structured execution evidence graph. Its agents should summarize, diagnose, correlate, and recommend without becoming the authoritative system of record.

5. Recommended Data Architecture

The strongest storage architecture for SNAP Qube is PostgreSQL with PostGIS as the authoritative relational and geospatial system of record. The platform should supplement PostgreSQL/PostGIS with time-series partitioning, Redis, event streaming, and object storage.

Storage Layer	Recommended Technology	Purpose
Operational DB	PostgreSQL	Users, roles, teams, devices, drive plans, sessions
Geospatial DB	PostGIS extension	Routes, polygons, roadway paths, RAN assets, GPS traces
Time-Series Layer	PostgreSQL partitioning or Timescale-style hypertables	High-volume telemetry and KPI events
Object Storage	S3-compatible storage	APK logs, generated reports, exports, screenshots, large artifacts
Cache / Session State	Redis	Live execution state, workspace persistence, command state, temporary streams
Event Bus	NATS, Kafka, or RabbitMQ	Device telemetry, command events, execution events, AI jobs
AI Memory / Trace Store	PostgreSQL + vector index where required	Agent outputs, embeddings, report context, analysis history
Search Index	OpenSearch or PostgreSQL full-text search	Reports, devices, plans, logs, AI summaries

Database Decision

MongoDB may be useful for flexible AI payloads or temporary agent memory, but it should not be the primary operational database for drive plans, route geometry, team hierarchy, RBAC, device inventory, or RAN geospatial analysis.

6. RBAC and Visibility Model

SNAP Qube requires role-based access control across the portal and backend APIs. Authorization must be enforced both in the React UI and server-side service layer, with Team/Subteam hierarchy applied to every query and action.

6.1 Role Responsibilities

Role	Primary Responsibility
SYSADM	Full platform administration across all teams, tenants, devices, infrastructure, users, and system policy
TEAMADM	Administrative control over a Team and its subteam hierarchy
SUBADM	Administrative control over one or more assigned Subteams
PLANNER	Creates, modifies, versions, and publishes drive plans
TESTER	Coordinates live drive-plan execution with field technicians and autonomous devices
VIEWER	Reviews plans, executions, reports, replays, dashboards, and AI summaries

6.2 Role Scope Matrix

Capability	SYSADM	TEAMADM	SUBADM	PLANNER	TESTER	VIEWER
Manage global platform settings	Yes	No	No	No	No	No
Manage Teams	Yes	Limited	No	No	No	No
Manage Subteams	Yes	Yes	Scoped	No	No	No
Manage users	Yes	Team-scoped	Subteam-scope d	No	No	No
Assign roles	Yes	Team-scoped	Subteam-scope d	No	No	No
Register devices	Yes	Yes	Yes	No	No	No
Assign devices to Team/Subteam	Yes	Team-scoped	Subteam-scope d	No	No	No
Manage APK versions	Yes	Optional view	No	No	No	No
Create drive plans	Yes	Yes	Scoped	Yes	No	No
Edit draft drive plans	Yes	Yes	Scoped	Yes	No	No
Publish drive plans	Yes	Yes	Scoped approval	Optional	No	No
Execute drive plans	Yes	Yes	Yes	Optional	Yes	No
View live execution	Yes	Yes	Yes	Yes	Yes	Yes
Stop execution	Yes	Yes	Scoped	Optional	Yes	No
View reports	Yes	Yes	Yes	Yes	Yes	Yes
Generate AI summary	Yes	Yes	Yes	Yes	Yes	View only
Export reports	Yes	Yes	Yes	Yes	Yes	Configurable
View audit logs	Yes	Team-scoped	Subteam-scope d	No	No	No

7. React Qube Portal Experience

The Qube portal should be a React-based workspace using purple as the primary accent color on a clean white background. It should not behave like a conventional CRUD application. The UX should support multiple concurrent canvas objects so users can work with maps, device panels, telemetry charts, plan editors, AI summaries, replay timelines, RAN infrastructure views, and execution dashboards without losing context.

UX Principle	Implementation Direction
Persistent Workspace	Open views, selected records, filters, map context, and unsaved edits persist across navigation.
Canvas-Oriented Interaction	Maps, charts, plan editors, reports, device views, and AI panels can be opened, docked, resized, minimized, and restored.
Role-Specific Landing Views	SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, and VIEWER see appropriate primary workflows immediately after login.
Map-First Execution	Live execution centers on route progress, device movement, RAN antennas, scoring overlays, and anomaly events.
Evidence-Docked AI	AI summaries should appear beside the evidence they reference rather than as disconnected chatbot text.

UX Principle	Implementation Direction
Replay as Analysis Tool	Completed drive plans must be replayable with synchronized map movement, test events, telemetry charts, notes, and anomalies.

8. Jira Epic Portfolio

The SNAP Qube implementation plan is organized into 26 Jira epics and 284 implementation stories. The portfolio is intended to guide design, implementation, testing, verification, deployment, documentation, and field pilot readiness.

Epic	Capability Area	Story Count	Primary Outcome
SQ-001	Product Foundation and System Architecture	8	Define the foundational architecture, domain model, service boundaries, integration contracts, and delivery standards for SNAP Qube.
SQ-002	Team, Subteam, and Tenant Hierarchy Management	9	Support hierarchical organizational modeling where Teams contain Subteams, and Subteams may contain nested Subteams.
SQ-003	Identity, Authentication, Authorization, and RBAC	10	Implement secure role-based portal and API access for SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, and VIEWER.
SQ-004	Autonomous Android Device Registry	10	Register, identify, classify, and manage Autonomous Android devices running the Drayvn Autonomous Android APK.
SQ-005	Autonomous Android Controller Command Plane	10	Build the controller service responsible for commanding, regulating, and supervising Autonomous Android devices.
SQ-006	APK Lifecycle, OTA Updates, and Device Log Retrieval	10	Allow SNAP Qube to manage APK versions, device updates, rollbacks, and remote log collection.
SQ-007	Drive Plan Authoring and CRUD	13	Allow PLANNER users to create, edit, version, validate, publish, archive, and clone Drive Plans.
SQ-008	RAN Infrastructure Inventory and Map Layers	10	Provide geospatial visualization and querying of RAN infrastructure assets.
SQ-009	Test Profile and Application Workload Catalog	12	Define reusable end-user test profiles for calling, video, music, messaging, upload/download, and browser responsiveness.
SQ-010	Live Drive Execution Orchestration	12	Allow TESTER users to coordinate live execution of Drive Plans with field technicians and autonomous devices.
SQ-011	Real-Time Telemetry Ingestion	10	Ingest high-frequency RAN metrics, GPS telemetry, application test events, and device state from autonomous devices.
SQ-012	Real-Time Streaming to the Qube Portal	10	Provide real-time visualization streams from the Autonomous Android Controller to the React portal.
SQ-013	React Qube Portal Foundation	10	Build the core React-based user interface with purple accent, white background, role-specific views, and persistent context.
SQ-014	Planner Experience and Drive Plan Canvas	12	Deliver a "WOW" planning experience for creating and refining drive plans.
SQ-015	Tester Live Execution Experience	12	Provide a real-time control and monitoring experience for TESTER users during active drive plan execution.
SQ-016	RAN and Application Scoring Engine	12	Compute meaningful quality scores for RAN performance, route segments, application tests, and execution outcomes.
SQ-017	Drayvn AI IQ Layer and Agentic Analysis	15	Use Drayvn AI agents to analyze collected data, detect issues, explain outcomes, and generate summaries.
SQ-018	Post-Execution Reporting	12	Generate detailed execution reports containing metrics, maps, graphs, AI analysis, evidence, and recommendations.
SQ-019	Drive Plan Replay	10	Allow users to replay completed drive plan executions as if they were watching the live drive again.
SQ-020	Alerts, Notifications, and Operational Exceptions	10	Notify users about execution risks, device issues, KPI breaches, and AI-generated concerns.
SQ-021	Backend API Gateway and Service Contracts	10	Expose consistent, secure APIs for all Qube portal and controller interactions.
SQ-022	Data Governance, Security, and Audit	10	Protect telemetry, device, user, execution, and AI data with enterprise-grade controls.
SQ-023	Observability, Monitoring, and Operations	10	Provide operational visibility into Qube services, device fleet, telemetry streams, AI jobs, and execution health.
SQ-024	Automated Testing and Verification	13	Validate the SNAP Qube ecosystem across unit, integration, system, field, performance, and security dimensions.
SQ-025	Deployment, CI/CD, and Environment Management	12	Deploy SNAP Qube reliably across development, staging, pilot, and production environments.

Epic	Capability Area	Story Count	Primary Outcome
SQ-026	Documentation, Training, and Operational Readiness	12	Ensure SNAP Qube can be used, supported, operated, and extended by engineering, field, admin, and support teams.

9. Detailed Jira Implementation Plan

Each epic below provides a development-ready capability workstream. Story tables are intentionally structured with a story key, concise story statement, and acceptance direction suitable for Jira import or manual backlog creation.

SQ-001: Product Foundation and System Architecture

Epic Goal

Define the foundational architecture, domain model, service boundaries, integration contracts, and delivery standards for SNAP Qube.

Story Key	Story	Acceptance Direction
SQ-001.1	Define SNAP Qube domain model	Domain objects for Team, Subteam, Device, Drive Plan, Execution, Telemetry, Report, and AI Analysis are documented and approved
SQ-001.2	Define microservice architecture	Service boundary diagram, API responsibilities, event flows, and data ownership are documented
SQ-001.3	Define canonical event model	Device events, telemetry events, command events, execution events, and AI analysis events have normalized schemas
SQ-001.4	Define data storage strategy	PostgreSQL/PostGIS, time-series partitioning, Redis, object storage, and event bus responsibilities are documented
SQ-001.5	Define API standards	REST/WebSocket/SSE conventions, error responses, pagination, auth headers, trace IDs, and OpenAPI rules are established
SQ-001.6	Define common audit model	All user, system, controller, device, and AI actions have standard audit event fields
SQ-001.7	Define platform naming and terminology	Product vocabulary is standardized across UI, APIs, Jira, documentation, and reports
SQ-001.8	Define MVP and release sequencing	MVP, pilot, production, and advanced AI milestones are documented

SQ-002: Team, Subteam, and Tenant Hierarchy Management

Epic Goal

Support hierarchical organizational modeling where Teams contain Subteams, and Subteams may contain nested Subteams.

Story Key	Story	Acceptance Direction
SQ-002.1	Create Team entity model	Teams can be created, updated, archived, and queried by authorized admins
SQ-002.2	Create recursive Subteam model	Subteams can be nested under Teams or parent Subteams
SQ-002.3	Implement hierarchy pathing	Each Subteam supports ancestry, descendants, inherited visibility, and scoped querying
SQ-002.4	Create Team/Subteam admin UI	SYSADM, TEAMADM, and SUBADM users can manage scoped hierarchy
SQ-002.5	Implement hierarchy-aware API filtering	All APIs return only data visible to the authenticated user's Team/Subteam scope
SQ-002.6	Add Team/Subteam assignment to devices	Registered devices can be assigned to Team/Subteam ownership
SQ-002.7	Add Team/Subteam assignment to drive plans	Drive plans are owned by a Team/Subteam and inherit visibility rules
SQ-002.8	Add Team/Subteam assignment to executions	Execution sessions inherit the scope of their parent drive plan
SQ-002.9	Implement hierarchy audit events	All Team/Subteam changes are audit logged

SQ-003: Identity, Authentication, Authorization, and RBAC

Epic Goal

Implement secure role-based portal and API access for SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, and VIEWER.

Story Key	Story	Acceptance Direction
SQ-003.1	Implement user model	Users have identity, status, profile metadata, Team/Subteam assignment, and role bindings
SQ-003.2	Implement authentication integration	Portal supports secure login using the selected identity provider or platform-native identity
SQ-003.3	Implement role model	Roles are defined as permission bundles aligned to SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, VIEWER
SQ-003.4	Implement scoped authorization middleware	APIs enforce role, Team, Subteam, and ownership restrictions
SQ-003.5	Implement portal route guards	React portal routes render only when role and scope allow access
SQ-003.6	Implement field-level UI permissions	Buttons, forms, actions, and exports are hidden or disabled based on role
SQ-003.7	Implement role assignment workflows	Admins can assign and revoke roles within their permitted scope
SQ-003.8	Implement impersonation-safe audit	All sensitive admin changes are logged with actor, target, scope, timestamp, and reason
SQ-003.9	Implement session timeout and token refresh	Portal sessions expire securely and refresh only when policy allows
SQ-003.10	Create RBAC verification suite	Automated tests confirm every role's access boundaries

SQ-004: Autonomous Android Device Registry

Epic Goal

Register, identify, classify, and manage Autonomous Android devices running the Drayvn Autonomous Android APK.

Story Key	Story	Acceptance Direction
SQ-004.1	Define device registration payload	APK reports device identity, OS version, APK version, radio capability, permissions, hardware profile, SIM/network info, and location capability
SQ-004.2	Implement device registration API	Controller or APK can register a device securely
SQ-004.3	Implement device approval workflow	Devices can be pending, approved, rejected, suspended, retired, or quarantined
SQ-004.4	Implement device inventory UI	Admins can view devices by Team, Subteam, status, APK version, health, and last heartbeat
SQ-004.5	Implement device assignment workflow	Devices can be assigned to Team/Subteam and optionally tagged by market, lab, field kit, or fleet
SQ-004.6	Implement device heartbeat tracking	Devices report online/offline, battery, network, GPS state, APK status, and command readiness
SQ-004.7	Implement device capability model	Portal can determine which tests each device supports
SQ-004.8	Implement duplicate registration handling	Re-registration updates existing device records without creating duplicates
SQ-004.9	Implement device quarantine	Devices with invalid identity, stale APK, failed integrity, or bad telemetry can be quarantined
SQ-004.10	Implement device audit trail	Registration, assignment, status, and command history is available to admins

SQ-005: Autonomous Android Controller Command Plane

Epic Goal

Build the controller service responsible for commanding, regulating, and supervising Autonomous Android devices.

Story Key	Story	Acceptance Direction
SQ-005.1	Define controller-to-device protocol	Commands, acknowledgments, errors, retries, timeouts, and state transitions are documented
SQ-005.2	Implement device command API	Backend can issue commands to start, stop, pause, resume, collect logs, update APK, and report status
SQ-005.3	Implement command queueing	Commands are queued and correlated to device acknowledgment
SQ-005.4	Implement command timeout handling	Timed-out commands are marked failed and visible in the portal
SQ-005.5	Implement command retry policy	Retry behavior is configurable by command type
SQ-005.6	Implement command cancellation	Pending commands can be canceled by authorized roles
SQ-005.7	Implement controller health dashboard	Admins can view connected devices, command latency, failure rates, and stream health
SQ-005.8	Implement execution start command	Controller can start drive-plan telemetry and test execution on selected devices
SQ-005.9	Implement execution stop command	Controller can safely terminate drive-plan execution and flush telemetry
SQ-005.10	Implement controller audit logging	Every command is traceable to user, drive plan, execution, device, and timestamp

SQ-006: APK Lifecycle, OTA Updates, and Device Log Retrieval

Epic Goal

Allow SNAP Qube to manage APK versions, device updates, rollbacks, and remote log collection.

Story Key	Story	Acceptance Direction
SQ-006.1	Create APK version catalog	SYSADM can register APK versions with version, checksum, release notes, compatibility, and rollout policy
SQ-006.2	Implement APK eligibility rules	Devices can be matched to required, optional, blocked, or deprecated APK versions
SQ-006.3	Implement OTA update command	Controller can instruct devices to download and install approved APK versions
SQ-006.4	Implement staged rollout	APK updates can target lab devices, subteams, regions, or percentages
SQ-006.5	Implement rollback support	Admins can initiate rollback to a prior approved APK version
SQ-006.6	Implement update status tracking	Portal shows pending, downloading, installing, installed, failed, rollback-required
SQ-006.7	Implement APK integrity validation	Device reports checksum/install validation after update
SQ-006.8	Implement remote APK log pull	Authorized users can pull APK logs from a device
SQ-006.9	Implement execution-specific log capture	Logs are associated with execution session and device
SQ-006.10	Implement log artifact viewer	Logs can be searched, filtered, downloaded, and referenced by AI analysis

SQ-007: Drive Plan Authoring and CRUD

Epic Goal

Allow PLANNER users to create, edit, version, validate, publish, archive, and clone Drive Plans.

Story Key	Story	Acceptance Direction
SQ-007.1	Define Drive Plan schema	Drive Plans include name, objective, Team/Subteam, route, area, infrastructure focus, test suite, devices, thresholds, schedule, and notes
SQ-007.2	Implement Drive Plan CRUD APIs	Authorized users can create, read, update, archive, and clone Drive Plans
SQ-007.3	Implement draft/published lifecycle	Plans support Draft, Review, Published, Scheduled, Executing, Completed, Archived
SQ-007.4	Implement versioning	Published plans are immutable and edits create a new version
SQ-007.5	Implement route drawing tools	Planner can draw or import roadway paths using map tools
SQ-007.6	Implement area-of-concentration tools	Planner can define polygon, corridor, radius, or manually selected area
SQ-007.7	Implement waypoint management	Planner can add, reorder, delete, annotate, and validate waypoints
SQ-007.8	Implement RAN asset selection	Planner can associate antennas, sectors, cells, or infrastructure groups with the plan
SQ-007.9	Implement device selection	Planner can reserve eligible devices for a plan
SQ-007.10	Implement test profile selection	Planner can define calling, media, messaging, upload/download, and browser test sequences
SQ-007.11	Implement plan validation	Portal validates missing route, no devices, unavailable test profiles, invalid thresholds, or out-of-scope assets
SQ-007.12	Implement plan cloning	Users can clone prior plans into new drafts
SQ-007.13	Implement plan export	Drive Plan can be exported as PDF/JSON for field review

SQ-008: RAN Infrastructure Inventory and Map Layers

Epic Goal

Provide geospatial visualization and querying of RAN infrastructure assets.

Story Key	Story	Acceptance Direction
SQ-008.1	Define RAN asset model	Sites, antennas, sectors, cells, bands, carriers, PCIs, TACs, azimuth, beamwidth, and technology are modeled
SQ-008.2	Implement RAN asset import	Admins can import infrastructure data from CSV, GeoJSON, API, or approved source format
SQ-008.3	Implement geospatial indexing	Assets are searchable by route, polygon, radius, Team/Subteam, and technology
SQ-008.4	Implement map overlay rendering	Portal displays RAN antennas, sectors, and coverage indicators on map

Story Key	Story	Acceptance Direction
SQ-008.5	Implement infrastructure filtering	Users can filter by LTE, NR, band, carrier, site, PCI, sector, market, or status
SQ-008.6	Implement infrastructure association to Drive Plans	Planner can bind target infrastructure to Drive Plan
SQ-008.7	Implement nearest-asset correlation	Telemetry events can be correlated to nearby RAN infrastructure
SQ-008.8	Implement serving-cell correlation	Metrics can be associated with serving and neighboring cells
SQ-008.9	Implement RAN asset history	Imported assets retain source, version, and timestamp
SQ-008.10	Implement RAN data validation	Invalid coordinates, duplicate assets, missing IDs, and malformed sectors are flagged

SQ-009: Test Profile and Application Workload Catalog

Epic Goal

Define reusable end-user test profiles for calling, video, music, messaging, upload/download, and browser responsiveness.

Story Key	Story	Acceptance Direction
SQ-009.1	Define test profile schema	Test profiles include name, type, duration, parameters, pass/fail thresholds, required permissions, and supported devices
SQ-009.2	Implement voice call test profile	Supports call setup, call duration, failure detection, drop detection, and quality markers
SQ-009.3	Implement SMS test profile	Supports send/receive timing, delivery confirmation, and failure capture
SQ-009.4	Implement MMS test profile	Supports media send/receive validation, latency, and failure reason
SQ-009.5	Implement video playback test profile	Supports startup delay, buffering, resolution changes, playback stall, and session quality
SQ-009.6	Implement music playback test profile	Supports startup delay, continuity, stall detection, and playback status
SQ-009.7	Implement upload/download test profile	Supports throughput, latency, jitter, failure, and server endpoint metadata
SQ-009.8	Implement browser responsiveness test profile	Supports page load time, first response, rendering completion, and timeout
SQ-009.9	Implement test sequencing	Drive Plans can execute profiles sequentially, cyclically, or conditionally
SQ-009.10	Implement test profile permissions	Only authorized users can create or modify shared test profiles
SQ-009.11	Implement profile versioning	Published profiles are immutable and referenced by execution session
SQ-009.12	Implement test profile simulator	Profiles can be validated without live field execution

SQ-010: Live Drive Execution Orchestration

Epic Goal

Allow TESTER users to coordinate live execution of Drive Plans with field technicians and autonomous devices.

Story Key	Story	Acceptance Direction
SQ-010.1	Implement execution session model	Execution records include Drive Plan version, tester, devices, route, start/end time, status, and telemetry stream IDs
SQ-010.2	Implement execution readiness check	System verifies devices online, GPS enabled, APK current, permissions valid, battery acceptable, and test profiles supported
SQ-010.3	Implement execution start workflow	TESTER can start a Drive Plan execution from the portal
SQ-010.4	Implement execution pause/resume	TESTER can pause or resume execution when supported
SQ-010.5	Implement execution stop workflow	TESTER can stop execution and trigger data finalization
SQ-010.6	Implement route deviation detection	Portal detects when GPS path deviates from planned route beyond configured tolerance
SQ-010.7	Implement test status timeline	Live view shows active test, queued test, completed test, failed test, and retry status
SQ-010.8	Implement multi-device execution	One execution can coordinate multiple autonomous devices
SQ-010.9	Implement field technician notes	TESTER can annotate live execution with observed field conditions
SQ-010.10	Implement execution exception handling	Device disconnect, GPS loss, command failure, app failure, or route deviation is visible and logged
SQ-010.11	Implement execution finalization	System locks telemetry, closes command stream, generates replay track, and triggers reporting pipeline
SQ-010.12	Implement execution audit	Start, stop, pause, resume, command, and override actions are traceable

SQ-011: Real-Time Telemetry Ingestion

Epic Goal

Ingest high-frequency RAN metrics, GPS telemetry, application test events, and device state from autonomous devices.

Story Key	Story	Acceptance Direction
SQ-011.1	Define telemetry payload schema	GPS, timestamp, device, session, radio metrics, app event, and network context are normalized
SQ-011.2	Implement telemetry ingestion API	Devices/controller can submit telemetry securely and efficiently
SQ-011.3	Implement streaming ingestion path	High-volume telemetry is accepted via event bus or streaming protocol
SQ-011.4	Implement telemetry validation	Invalid, stale, duplicate, malformed, or out-of-order telemetry is flagged
SQ-011.5	Implement telemetry partitioning	Data is partitioned by Team/Subteam, execution, device, and time
SQ-011.6	Implement GPS trace persistence	GPS points are stored with geospatial indexing
SQ-011.7	Implement RAN metric persistence	RSRP, RSRQ, SINR, RSSI, PCI, TAC, band, carrier, serving cell, neighbor cell, throughput, and latency are stored
SQ-011.8	Implement application test event persistence	Call, SMS, MMS, video, music, browser, upload, and download test events are stored
SQ-011.9	Implement telemetry backpressure handling	System degrades gracefully under burst load
SQ-011.10	Implement telemetry replay readiness	Stored telemetry supports deterministic replay by timestamp

SQ-012: Real-Time Streaming to the Qube Portal

Epic Goal

Provide real-time visualization streams from the Autonomous Android Controller to the React portal.

Story Key	Story	Acceptance Direction
SQ-012.1	Define live stream event contracts	Portal receives device location, RAN metrics, test status, alerts, scoring, and command status
SQ-012.2	Implement WebSocket/SSE live stream	Portal can subscribe to execution streams
SQ-012.3	Implement stream authorization	Users can subscribe only to executions within their visibility scope
SQ-012.4	Implement stream reconnect	Portal reconnects without losing current execution context
SQ-012.5	Implement live map update stream	Device position, route progress, and telemetry markers update in real time
SQ-012.6	Implement live KPI stream	Current scores and metrics update without page refresh
SQ-012.7	Implement stream compression or throttling	High-frequency telemetry is downsampled for UI display while preserving raw data
SQ-012.8	Implement stream health indicator	UI shows live, delayed, reconnecting, stale, or disconnected state
SQ-012.9	Implement multi-device stream aggregation	Portal can display multiple devices in one execution view
SQ-012.10	Implement stream audit and diagnostics	Stream sessions are observable and traceable

SQ-013: React Qube Portal Foundation

Epic Goal

Build the core React-based user interface with purple accent, white background, role-specific views, and persistent context.

Story Key	Story	Acceptance Direction
SQ-013.1	Create design system	Purple primary accent, white background, typography, spacing, cards, panels, icons, and status colors are defined
SQ-013.2	Create shell layout	Portal has global navigation, user menu, Team/Subteam selector, notifications, and workspace area
SQ-013.3	Implement role-specific home pages	SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, and VIEWER receive appropriate landing views
SQ-013.4	Implement workspace state persistence	Open pages, selected records, filters, draft edits, and map context survive navigation
SQ-013.5	Implement canvas object framework	Maps, charts, editors, reports, AI panels, and device views can open as concurrent canvas objects
SQ-013.6	Implement canvas docking and layout	Users can move, resize, minimize, maximize, and restore canvas objects
SQ-013.7	Implement unsaved changes protection	Users are warned before losing unsaved Drive Plan edits
SQ-013.8	Implement global search	Users can search drive plans, executions, devices, reports, and RAN assets within scope

Story Key	Story	Acceptance Direction
SQ-013.9	Implement notification center	Device alerts, execution events, report completion, and AI findings appear in portal
SQ-013.10	Implement accessibility baseline	UI supports keyboard navigation, contrast compliance, focus states, and screen-reader-safe labels

SQ-014: Planner Experience and Drive Plan Canvas

Epic Goal

Deliver a “WOW” planning experience for creating and refining drive plans.

Story Key	Story	Acceptance Direction
SQ-014.1	Implement Drive Plan list view	PLANNER can view, filter, search, sort, clone, and open plans
SQ-014.2	Implement Drive Plan editor canvas	Plan metadata, route, area, infrastructure, tests, devices, and thresholds are edited in one workspace
SQ-014.3	Implement route map editor	Planner can draw, edit, import, and validate routes
SQ-014.4	Implement area editor	Planner can draw and edit concentration zones
SQ-014.5	Implement RAN overlay selector	Planner can toggle RAN assets while designing routes
SQ-014.6	Implement test sequence builder	Planner can drag, order, configure, and validate application tests
SQ-014.7	Implement device eligibility panel	Planner sees available devices compatible with selected tests
SQ-014.8	Implement threshold builder	Planner defines KPI thresholds for route, test, RAN, and application outcomes
SQ-014.9	Implement AI-assisted plan review	Drayvn AI reviews plan completeness, risk, route gaps, and infrastructure coverage
SQ-014.10	Implement plan publish workflow	Planner can submit, validate, and publish drive plan versions
SQ-014.11	Implement plan compare view	Users can compare versions of a drive plan
SQ-014.12	Implement plan readiness score	Portal calculates whether a plan is execution-ready

SQ-015: Tester Live Execution Experience

Epic Goal

Provide a real-time control and monitoring experience for TESTER users during active drive plan execution.

Story Key	Story	Acceptance Direction
SQ-015.1	Implement execution launch page	TESTER can select a published Drive Plan and verify readiness
SQ-015.2	Implement live execution dashboard	Dashboard shows route, current GPS, RAN assets, tests, scores, devices, and alerts
SQ-015.3	Implement real-time map	Map shows vehicle/device movement, route progress, telemetry coloring, and antenna overlays
SQ-015.4	Implement RAN metric strip	Live view shows key radio metrics and quality indicators
SQ-015.5	Implement application test panel	Live view shows active calling, media, SMS/MMS, throughput, and browser tests
SQ-015.6	Implement device health panel	Battery, signal, GPS, APK, connection, and command status are visible
SQ-015.7	Implement route progress indicator	Portal shows percentage complete, next segment, missed segment, and route deviation
SQ-015.8	Implement live scoring overlay	Route segments are colored by quality score
SQ-015.9	Implement anomaly feed	Live anomalies are shown as timestamped events
SQ-015.10	Implement manual note capture	TESTER can add notes tied to timestamp and map location
SQ-015.11	Implement execution controls	Start, stop, pause, resume, and emergency stop are role-protected
SQ-015.12	Implement post-stop transition	When execution completes, portal transitions to summary generation state

SQ-016: RAN and Application Scoring Engine

Epic Goal

Compute meaningful quality scores for RAN performance, route segments, application tests, and execution outcomes.

Story Key	Story	Acceptance Direction
SQ-016.1	Define scoring model	Scoring dimensions include coverage, stability, throughput, latency, application behavior, test success, and route compliance
SQ-016.2	Implement segment-level scoring	Route is divided into segments with individual scores
SQ-016.3	Implement RAN signal scoring	RSRP, RSRQ, SINR, PCI changes, serving-cell stability, and neighbor behavior contribute to score
SQ-016.4	Implement throughput scoring	Upload/download test outcomes contribute to performance score
SQ-016.5	Implement calling score	Call setup, drop, duration, and failure events contribute to voice score
SQ-016.6	Implement messaging score	SMS/MMS latency, delivery, and failures contribute to messaging score
SQ-016.7	Implement video playback score	Startup delay, stalls, buffering, resolution, and continuity contribute to video score
SQ-016.8	Implement music playback score	Startup delay, stalls, continuity, and failure events contribute to media score
SQ-016.9	Implement browser score	Page load timing, timeout, responsiveness, and failure contribute to browser score
SQ-016.10	Implement composite execution score	Overall execution score combines route, RAN, application, and device confidence
SQ-016.11	Implement confidence score	System indicates whether data quality is sufficient for analysis
SQ-016.12	Implement scoring explainability	Scores expose contributing factors and raw evidence references

SQ-017: Drayvn AI IQ Layer and Agentic Analysis

Epic Goal

Use Drayvn AI agents to analyze collected data, detect issues, explain outcomes, and generate summaries.

Recommended Agentic Stack

Use LangGraph for agent orchestration and stateful workflows, with Langfuse or equivalent observability for tracing prompts, agent decisions, evaluation, and model behavior.

Story Key	Story	Acceptance Direction
SQ-017.1	Define AI agent architecture	Agent roles, tools, memory, data access boundaries, and orchestration graph are documented
SQ-017.2	Implement AI job queue	Completed executions can trigger asynchronous AI analysis jobs
SQ-017.3	Implement Execution Summary Agent	Agent generates plain-language summary of drive execution
SQ-017.4	Implement RAN Anomaly Agent	Agent identifies likely coverage, handoff, signal, congestion, or interference anomalies
SQ-017.5	Implement Application Experience Agent	Agent analyzes call, video, music, messaging, browser, and throughput test behavior
SQ-017.6	Implement Route Compliance Agent	Agent evaluates whether the drive followed the planned route and area
SQ-017.7	Implement Device Health Agent	Agent identifies whether device state affected data quality
SQ-017.8	Implement Infrastructure Correlation Agent	Agent correlates issues to nearby RAN infrastructure assets
SQ-017.9	Implement Recommendation Agent	Agent produces engineering recommendations and next-step actions
SQ-017.10	Implement AI evidence binding	Every AI conclusion links to supporting telemetry, KPI, map segment, or artifact
SQ-017.11	Implement AI confidence levels	Agent output includes confidence, uncertainty, and data sufficiency assessment
SQ-017.12	Implement AI safety and guardrails	Agents cannot fabricate infrastructure facts or unsupported conclusions
SQ-017.13	Implement AI trace viewer	Authorized users can inspect analysis stages, tool calls, and source evidence
SQ-017.14	Implement AI regenerate workflow	Authorized users can regenerate summary after data correction or threshold change
SQ-017.15	Implement AI feedback loop	Users can rate AI findings as useful, incorrect, incomplete, or needs review

SQ-018: Post-Execution Reporting

Epic Goal

Generate detailed execution reports containing metrics, maps, graphs, AI analysis, evidence, and recommendations.

Story Key	Story	Acceptance Direction
SQ-018.1	Define report structure	Report includes executive summary, plan metadata, route, devices, tests, KPIs, anomalies, AI findings, charts, and recommendations
SQ-018.2	Implement report generation pipeline	Completed executions automatically trigger report generation

Story Key	Story	Acceptance Direction
SQ-018.3	Implement report dashboard	Users can view reports by Team/Subteam, plan, execution, date, status, and score
SQ-018.4	Implement route heatmap section	Report shows route colored by quality score
SQ-018.5	Implement RAN metric charts	Report includes signal, throughput, latency, handoff, and application charts
SQ-018.6	Implement application test summary	Report summarizes call, SMS/MMS, video, music, browser, upload, and download outcomes
SQ-018.7	Implement anomaly table	Report lists anomalies with timestamp, location, metric, impact, and evidence
SQ-018.8	Implement AI summary section	Report includes Drayvn AI explanation and recommendations
SQ-018.9	Implement device data quality section	Report identifies GPS gaps, device disconnects, stale data, or APK issues
SQ-018.10	Implement PDF export	Report can be exported to PDF
SQ-018.11	Implement CSV/JSON export	Raw or summarized data can be exported by authorized users
SQ-018.12	Implement report versioning	Regenerated reports maintain version history

SQ-019: Drive Plan Replay

Epic Goal

Allow users to replay completed drive plan executions as if they were watching the live drive again.

Story Key	Story	Acceptance Direction
SQ-019.1	Define replay timeline model	Replay supports synchronized GPS, RAN metrics, test events, alerts, notes, and scoring
SQ-019.2	Implement replay data preparation	Completed telemetry is transformed into replay-ready timeline data
SQ-019.3	Implement replay UI canvas	Users can replay route movement, metric changes, test execution, and anomalies
SQ-019.4	Implement timeline controls	Replay supports play, pause, seek, speed control, jump to anomaly, and jump to test
SQ-019.5	Implement replay map overlay	Route, GPS position, RAN assets, scores, and telemetry markers render during replay
SQ-019.6	Implement synchronized charts	Charts update based on replay timestamp
SQ-019.7	Implement anomaly markers	Users can jump directly to AI-detected or system-detected anomalies
SQ-019.8	Implement notes overlay	Field notes appear at the correct timestamp and map location
SQ-019.9	Implement multi-device replay	Replay supports multiple devices in the same execution
SQ-019.10	Implement replay export reference	Reports can link to replay timestamps

SQ-020: Alerts, Notifications, and Operational Exceptions

Epic Goal

Notify users about execution risks, device issues, KPI breaches, and AI-generated concerns.

Story Key	Story	Acceptance Direction
SQ-020.1	Define alert taxonomy	Alerts include device, execution, telemetry, route, RAN, application, system, and AI categories
SQ-020.2	Implement live execution alerts	TESTER sees alerts during active execution
SQ-020.3	Implement device health alerts	Low battery, GPS disabled, stale heartbeat, APK mismatch, or lost connection triggers alert
SQ-020.4	Implement KPI threshold alerts	Route segment or test threshold breaches trigger visible alert
SQ-020.5	Implement route deviation alerts	Drive route deviation appears in live dashboard
SQ-020.6	Implement telemetry gap alerts	Missing or stale data is flagged
SQ-020.7	Implement notification center	Users can view unread, active, resolved, and historical alerts
SQ-020.8	Implement alert acknowledgment	Users can acknowledge, resolve, or comment on alerts
SQ-020.9	Implement AI-generated alert recommendations	AI can suggest likely cause and recommended action
SQ-020.10	Implement alert audit events	Alert lifecycle is traceable

SQ-021: Backend API Gateway and Service Contracts

Epic Goal

Expose consistent, secure APIs for all Qube portal and controller interactions.

Story Key	Story	Acceptance Direction
SQ-021.1	Implement API gateway	Gateway routes requests to backend services with auth and tracing
SQ-021.2	Implement OpenAPI specifications	All REST APIs are documented and published
SQ-021.3	Implement API versioning	APIs support stable version paths
SQ-021.4	Implement standardized error model	All APIs return consistent error codes and diagnostic correlation IDs
SQ-021.5	Implement pagination and filtering	List APIs support role-aware pagination, filtering, and sorting
SQ-021.6	Implement request tracing	Requests carry trace ID across services
SQ-021.7	Implement rate limiting	APIs protect against runaway portal, device, or integration calls
SQ-021.8	Implement service-to-service authentication	Internal calls use secure credentials or mTLS-style trust
SQ-021.9	Implement API contract tests	API behavior is validated against OpenAPI specs
SQ-021.10	Implement developer API documentation	Internal developers can test APIs with examples

SQ-022: Data Governance, Security, and Audit

Epic Goal

Protect telemetry, device, user, execution, and AI data with enterprise-grade controls.

Story Key	Story	Acceptance Direction
SQ-022.1	Define data classification model	User, device, GPS, telemetry, RAN, reports, logs, and AI outputs are classified
SQ-022.2	Implement encryption at rest	Sensitive data stores use encryption
SQ-022.3	Implement encryption in transit	Portal, API, controller, and device communication uses secure transport
SQ-022.4	Implement audit event storage	Security-relevant actions are stored immutably or append-only
SQ-022.5	Implement data retention policy	Telemetry, logs, reports, and artifacts have configurable retention
SQ-022.6	Implement scoped export permissions	Export actions require proper role and scope
SQ-022.7	Implement artifact access control	Logs and reports are visible only to authorized users
SQ-022.8	Implement secrets management	API keys, signing keys, device tokens, and OTA credentials are securely managed
SQ-022.9	Implement device identity hardening	Device registration requires secure identity validation
SQ-022.10	Implement security test suite	RBAC, auth, API, and data access controls are tested

SQ-023: Observability, Monitoring, and Operations

Epic Goal

Provide operational visibility into Qube services, device fleet, telemetry streams, AI jobs, and execution health.

Story Key	Story	Acceptance Direction
SQ-023.1	Define observability standards	Logs, metrics, traces, health checks, and alerts are standardized
SQ-023.2	Implement service health checks	Each microservice exposes liveness and readiness status
SQ-023.3	Implement telemetry ingestion metrics	Monitor event rate, validation failure rate, lag, and drop rate
SQ-023.4	Implement controller metrics	Monitor device connections, command latency, failure rate, and OTA status
SQ-023.5	Implement portal usage metrics	Monitor page performance, stream performance, and user actions
SQ-023.6	Implement AI job observability	Monitor AI queue time, execution time, failure rate, token usage, and confidence distribution
SQ-023.7	Implement execution monitoring dashboard	Ops users can view active executions and system state
SQ-023.8	Implement alerting rules	Operations alerts trigger when system health degrades
SQ-023.9	Implement log correlation	Logs are correlated by execution ID, device ID, user ID, and trace ID

Story Key	Story	Acceptance Direction
SQ-023.10	Implement runbooks	Support procedures exist for common failures

SQ-024: Automated Testing and Verification

Epic Goal

Validate the SNAP Qube ecosystem across unit, integration, system, field, performance, and security dimensions.

Story Key	Story	Acceptance Direction
SQ-024.1	Define test strategy	Test plan covers portal, APIs, controller, device, telemetry, AI, replay, reports, and RBAC
SQ-024.2	Implement unit tests	Core services meet coverage threshold
SQ-024.3	Implement API integration tests	Drive Plan, device, execution, telemetry, and report APIs are tested
SQ-024.4	Implement RBAC test matrix	Each role is tested against allowed and denied actions
SQ-024.5	Implement telemetry load tests	System handles expected telemetry volume with acceptable latency
SQ-024.6	Implement live stream tests	Portal receives live data under normal and degraded conditions
SQ-024.7	Implement execution simulator	Simulated devices can run fake drive plans for test automation
SQ-024.8	Implement replay verification	Replayed timeline matches ingested telemetry order
SQ-024.9	Implement AI output validation	AI summaries must cite evidence and pass hallucination guardrails
SQ-024.10	Implement report verification	Generated reports match execution data and scoring outputs
SQ-024.11	Implement OTA update tests	APK rollout, failure, retry, and rollback flows are validated
SQ-024.12	Implement security testing	Auth, permissions, export controls, and device registration are tested
SQ-024.13	Implement field pilot test plan	Controlled field drive validates end-to-end platform behavior

SQ-025: Deployment, CI/CD, and Environment Management

Epic Goal

Deploy SNAP Qube reliably across development, staging, pilot, and production environments.

Story Key	Story	Acceptance Direction
SQ-025.1	Define environment strategy	Dev, QA, staging, pilot, and production environment requirements are documented
SQ-025.2	Implement containerized services	Microservices are packaged consistently
SQ-025.3	Implement CI pipeline	Code builds, tests, scans, and packages automatically
SQ-025.4	Implement CD pipeline	Approved builds deploy to target environments
SQ-025.5	Implement database migration pipeline	Schema changes are versioned and repeatable
SQ-025.6	Implement configuration management	Environment-specific settings are externalized
SQ-025.7	Implement secrets deployment	Secrets are injected securely
SQ-025.8	Implement rollback strategy	Services and database migrations have documented rollback path
SQ-025.9	Implement blue/green or staged release	Production updates minimize downtime
SQ-025.10	Implement deployment verification	Smoke tests validate successful deployment
SQ-025.11	Implement infrastructure-as-code	Platform infrastructure is reproducible
SQ-025.12	Implement backup and restore	Databases and artifacts have tested backup/restore workflows

SQ-026: Documentation, Training, and Operational Readiness

Epic Goal

Ensure SNAP Qube can be used, supported, operated, and extended by engineering, field, admin, and support teams.

Story Key	Story	Acceptance Direction
SQ-026.1	Create product overview documentation	Explains SNAP Qube purpose, concepts, roles, and workflows
SQ-026.2	Create admin guide	Covers Teams, Subteams, users, roles, devices, APKs, and system settings

Story Key	Story	Acceptance Direction
SQ-026.3	Create planner guide	Covers drive plan creation, route design, RAN overlays, test selection, and publishing
SQ-026.4	Create tester guide	Covers live execution, field coordination, alerts, notes, stop/start, and post-run summary
SQ-026.5	Create viewer guide	Covers report review, replay, dashboards, and exports
SQ-026.6	Create API documentation	Documents backend APIs and integration patterns
SQ-026.7	Create device onboarding guide	Documents Android device preparation, APK installation, permissions, registration, and troubleshooting
SQ-026.8	Create controller operations guide	Documents controller deployment, command flow, logs, OTA, and diagnostics
SQ-026.9	Create AI analysis guide	Explains AI findings, confidence, evidence, and limitations
SQ-026.10	Create support runbooks	Covers common device, execution, stream, telemetry, and report failures
SQ-026.11	Create pilot onboarding package	Enables a Team/Subteam to run first controlled pilot
SQ-026.12	Create release notes template	Standardized release communication for portal, backend, controller, and APK

10. Suggested Release Plan

The recommended release plan sequences the system from foundation to field pilot readiness. This sequencing deliberately separates architecture and RBAC foundation, device and planning MVP, live execution, AI/reporting/replay, and enterprise hardening.

Release	Theme	Included Epics	Exit Criteria
Release 0	Architecture and Foundation	SQ-001, SQ-002, SQ-003, SQ-021, SQ-025	Domain model approved; auth/RBAC working; hierarchy operational; backend scaffolding deployed; React portal shell available
Release 1	Device Registration and Drive Plan MVP	SQ-004, SQ-006, SQ-007, SQ-008, SQ-013, SQ-014	Devices register in portal; Team/Subteam assignment works; plans can be created, validated, and published; RAN infrastructure appears on map
Release 2	Live Execution and Telemetry Streaming	SQ-005, SQ-010, SQ-011, SQ-012, SQ-015, SQ-020	TESTER can start/stop execution; devices stream GPS/RAN metrics; live route progress, device health, command state, and alerts are visible
Release 3	Scoring, Reporting, Replay, and AI	SQ-016, SQ-017, SQ-018, SQ-019, SQ-024	Execution generates score and report; replay works with synchronized telemetry; AI summary includes evidence-bound conclusions
Release 4	Enterprise Hardening and Pilot Readiness	SQ-022, SQ-023, SQ-024, SQ-025, SQ-026	Security verified; observability active; backup/restore tested; pilot guide complete; field pilot can run end-to-end

11. Global Definition of Done

Every Jira story should satisfy the following criteria unless explicitly waived by product and architecture leadership. This definition of done keeps implementation quality aligned across portal, backend, device, AI, data, security, and deployment workstreams.

Category	Done Criteria
Functional	Feature works for intended role and scope
API	API documented, versioned, validated, and covered by tests
Security	RBAC enforced server-side and UI-side
Audit	Important actions generate audit events
Data	Data model migration exists and is reversible where practical
UI	React view follows Qube design system
Testing	Unit and integration tests completed
Observability	Logs, metrics, and trace IDs added where appropriate
Error Handling	User-facing errors are clear and system errors are traceable
Documentation	Feature usage and operational notes documented
Performance	Meets agreed latency and scale expectations
Review	Code reviewed and accepted

Category	Done Criteria
Deployment	Works in target environment through CI/CD

12. Critical Product and Architecture Decisions

Decision Area	Recommended Position
Database / System of Record	Use PostgreSQL with PostGIS as the authoritative relational and geospatial data platform. Add partitioned time-series tables for telemetry, Redis for live state, and object storage for heavy artifacts.
AI Governance	Use Drayvn AI as an analysis and orchestration layer. AI may explain, summarize, recommend, and correlate, but must not become the source of truth or invent unsupported RAN facts.
UX Model	Build the portal as a persistent React workspace, not a flat CRUD interface. Use concurrent canvas objects, map-first execution, replay, and evidence-docked AI insight panels.
Execution Model	Treat the Autonomous Android Controller as the field control plane, responsible for device registration, APK OTA, command dispatch, live stream supervision, and log retrieval.

12.1 AI Guardrails

AI Agents Should	AI Agents Should Not
Analyze completed executions; explain anomalies; generate summaries; recommend next actions; correlate telemetry to infrastructure; highlight uncertainty; bind conclusions to evidence.	Invent RAN infrastructure facts; modify execution records without workflow; override scoring without traceable policy; produce unsupported root-cause claims.

13. MVP Scope Recommendation

The first serious MVP should be credible enough for a management demonstration, technically meaningful for engineering validation, and operationally realistic for a controlled field pilot. The following capability set is recommended as the initial MVP scope.

#	MVP Capability
1	Team/Subteam hierarchy
2	RBAC for SYSADM, TEAMADM, SUBADM, PLANNER, TESTER, and VIEWER
3	Device registration and inventory
4	Controller command path for start, stop, and log pull
5	Drive Plan CRUD with route and area definition
6	RAN infrastructure import and map overlay
7	Live execution dashboard
8	Real-time GPS and core RAN telemetry streaming
9	Basic test profile execution status
10	Post-execution report generation
11	Replay of GPS route and telemetry timeline
12	AI-generated execution summary with evidence references

MVP Outcome

This MVP would give Procal Technologies a credible, demonstrable, and commercially meaningful foundation for advanced autonomous testing, deeper RAN analytics, and Drayvn AI differentiation.

14. Recommended Next Actions

Priority	Action	Owner
1	Approve product scope and MVP boundaries using the release plan in this document.	Product + Executive Sponsor
2	Convert epic/story tables into Jira backlog items with labels for portal, backend, Android, AI, data, DevOps, QA, and security.	Product Owner + Scrum Lead
3	Finalize service boundary diagram, canonical event schemas, and API gateway standards before feature implementation accelerates.	Architecture Lead
4	Prototype the React canvas workspace and live execution map first, because they define the product experience.	UX Lead + Frontend Lead
5	Build a device simulator early to unblock telemetry ingestion, replay, scoring, reporting, and AI pipeline validation before field devices are fully available.	Backend Lead + QA Lead
6	Run a controlled pilot drive test using a limited Team/Subteam hierarchy, one drive plan, one controller, and a small autonomous Android device fleet.	Field Ops + Engineering

End of Report